

**HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ
VIỆN CÔNG NGHỆ THÔNG MINH VÀ TƯƠNG TÁC**



BÁO CÁO VÒNG MÔ PHỎNG

Họ và tên: Bùi Lê Uyên Vy - 31241027729

TP. Hồ Chí Minh, ngày 27 tháng 9 năm 2026

TÓM TẮT NGHIÊN CỨU

Nghiên cứu này trình bày giải pháp toàn diện cho bài toán điều khiển robot tự hành trong môi trường mô phỏng 3D Gazebo thuộc khuôn khổ cuộc thi UEH CRC 2026. Giải pháp được đóng gói hoàn chỉnh trong một nút tính toán duy nhất (`starter_node.py`) vận hành trên nền tảng Robot Operating System 2 (ROS 2 Humble/Foxy).

Hệ thống được chia thành hai khối chức năng độc lập: Khối Nhận thức (Perception) và Khối Điều khiển (Control). Khối Nhận thức xử lý luồng dữ liệu hình ảnh thô từ chủ đề `/camera/image_raw` tần số 30 Hz thông qua các kỹ thuật tiền xử lý: trích xuất Vùng quan tâm (ROI), làm mịn nhiễu Gauss, tăng cường độ tương phản cục bộ thích ứng (CLAHE), phân đoạn không gian màu HSV và tính toán các mô-men không gian (Spatial Moments) để xác định tâm làn đường cx. Đồng thời, dữ liệu hình ảnh kết hợp với đám mây điểm LiDAR từ `/scan` được hợp nhất để phát hiện chướng ngại vật màu tím (người đi bộ) và biển báo giao thông đỏ. Khối Điều khiển tiếp nhận sai lệch điểm ảnh $e(t)$ để tính toán vận tốc góc $w(t)$ dựa trên bộ điều khiển Vi-Tỷ-lệ (PD Controller) kết hợp cơ chế điều chỉnh tốc độ tiến $v(t)$ linh hoạt theo độ cong của đường và thuật toán xử lý mất vạch kẻ. Kết quả thực nghiệm cho thấy hệ thống vận hành ổn định tại chu kỳ 20 Hz, đạt độ trễ tính toán dưới 18 ms và đảm bảo robot hoàn thành sa bàn an toàn, mượt mà.

PHẦN 1: TỔNG QUAN PHƯƠNG PHÁP VÀ KIẾN TRÚC HỆ THỐNG

1.1. Triết lý thiết kế và Luồng xử lý dữ liệu hệ thống:

Trong các bài toán điều khiển robot tự hành mô phỏng thời gian thực, thách thức kỹ thuật lớn nhất là đảm bảo sự cân bằng giữa độ chính xác của thuật toán nhận thức thị giác và tần số đáp ứng động học của bộ điều khiển. Hệ thống tự hành trong nghiên cứu này được xây dựng theo mô hình xử lý tập trung trên một Nút ROS 2 (ROS 2 Node) duy nhất mang tên Starter, được định nghĩa trong tệp mã nguồn `starter_node.py`. Nút tính toán này vận hành theo cơ chế vòng lặp định thời tự động (Timer Callback) với tần số được thiết lập cố định là 20 Hz, tương ứng với chu kỳ thực thi là 50 ms cho mỗi vòng lặp tính toán thông qua câu lệnh `self.create_timer(1.0 / rate, self.tick)`.

Triết lý thiết kế cốt lõi của hệ thống tập trung vào việc phân tách độc lập và rành mạch giữa hai giai đoạn xử lý nối tiếp nhau:

1. Giai đoạn Nhận thức Thị giác (Perception Phase): Nhiệm vụ của giai đoạn này là tiếp nhận ma trận điểm ảnh màu BGR đầu vào từ camera có kích thước không gian $480 \times 640 \times 3$ pixels. Thông qua các phép biến đổi hình thái học, lọc màu và tính toán mô-men không gian, hệ thống sẽ nén toàn bộ dữ liệu hình ảnh thô khổng lồ đó thành một giá trị vô hướng đại biểu duy nhất phản ánh trạng thái hình học của xe trên đường chạy: Độ lệch vị trí giữa tâm xe và tâm làn đường lý thuyết, ký hiệu là:
$$e(t) = c_x - width/2$$
2. Giai đoạn Điều khiển Động học (Control Phase): Tiếp nhận giá trị sai lệch $e(t)$ thu được từ khối nhận thức, khối điều khiển sẽ áp dụng thuật toán vi-tỷ-lệ để tính toán vận tốc góc lái $w(t)$ (được gán cho tham số `angular.z`), đồng thời đưa ra quyết định vận tốc tiến thẳng $v(t)$ (được gán cho tham số `linear.x`). Cuối cùng, tín hiệu điều khiển này sẽ được đóng gói và xuất tới chủ đề giao tiếp `/cmd_vel` để truyền tới bộ chấp hành động cơ của robot.

1.2. Môi trường mô phỏng và Giao tiếp ROS 2 Topics:

Hệ thống tự hành liên lạc và trao đổi dữ liệu với môi trường mô phỏng không gian 3D Gazebo thông qua ba chủ đề truyền tin (ROS 2 Topics) chính với các quy chuẩn kiểu dữ liệu cụ thể như sau:

- Chủ đề `/camera/image_raw` (Kiểu tin nhắn `sensor_msgs/msg/Image`): Đây là kênh truyền tải luồng hình ảnh thô liên tục từ cảm biến camera ảo được gắn ở vị trí phía trước đầu xe. Dữ liệu được phát ra với tần số 30 Hz và độ phân giải không gian là 480×640 pixels. Nút tính toán sử dụng thư viện cầu nối chuyên dụng `Cv_Bridge` thông qua phương thức `self.bridge.imgmsg_to_cv2(msg, 'bgr8')` để chuyển đổi cấu trúc tin nhắn chuẩn của ROS thành ma trận đa chiều NumPy tương thích hoàn toàn với thư viện xử lý ảnh OpenCV.

- Chủ đề /scan (Kiểu tin nhắn sensor_msgs/msg/LaserScan): Kênh truyền tải mảng dữ liệu đo khoảng cách từ cảm biến quét laser 2D (LiDAR) xoay 360° quanh xe với tần số cập nhật 10 Hz. Nhằm phục vụ cho việc phát hiện chướng ngại vật phía trước, hệ thống xây dựng hàm hỗ trợ range_at(angle_deg, width_deg). Hàm này có chức năng bóc tách, lọc bỏ các giá trị không hợp lệ (như inf hoặc nhỏ hơn range_min) và tìm ra khoảng cách đo được ngắn nhất trong phạm vi hình quạt $\pm 12.5^\circ$ đối xứng qua trục chính giữa đầu xe (0°).
- Chủ đề /cmd_vel (Kiểu tin nhắn geometry_msgs/msg/Twist): Kênh tiếp nhận các lệnh điều khiển động học được xuất ra từ nút tính toán với tần số 20 Hz. Cấu trúc tin nhắn bao gồm hai thành phần vô hướng quan trọng: vận tốc tiến thẳng linear.x (được khống chế trong dải $[-\text{max_speed}, \text{max_speed}]$ với $\text{max_speed} = 0.12$ m/s) và vận tốc góc xoay quanh trục đứng angular.z (được khống chế trong dải $[-\text{max_turn}, \text{max_turn}]$ với $\text{max_turn} = 1.0$ rad/s).

PHẦN 2: THUẬT TOÁN XỬ LÝ ẢNH VÀ NHẬN THỨC

Toàn bộ quy trình xử lý hình ảnh được đóng gói bên trong hàm control() của nút starter_node.py. Mỗi khung hình nhận được từ camera đều trải qua chuỗi các bước biến đổi toán học và xử lý hình học nghiêm ngặt theo trình tự sau:

2.1. Cắt vùng quan tâm (ROI Cropping):

Trong khung hình ảnh gốc thu được từ camera có kích thước 480×640 pixels, 2/3 diện tích phía trên khung hình chủ yếu chứa cảnh quan môi trường xung quanh như bầu trời, các công trình sa bàn, tường chắn xa và biển báo không phục vụ trực tiếp cho nhiệm vụ bám làn. Nếu giữ lại toàn bộ khung hình, hệ thống sẽ bị lãng phí tài nguyên tính toán và dễ bị ảnh hưởng bởi các yếu tố nhiễu ánh sáng bên ngoài.

Do đó, thuật toán thực hiện trích xuất Vùng quan tâm (Region of Interest - ROI) bằng cách chỉ giữ lại 1/3 nửa dưới của bức ảnh, tính từ tọa độ dòng thứ 320 (tương ứng 2/3 chiều cao 480) đến dòng thứ 480:

```
ROI=Image[320:480,0:640]
```

Thao tác cắt vùng này thu hẹp kích thước ma trận cần xử lý xuống còn 160×640 pixels, giúp loại bỏ tới 66.7% số lượng điểm ảnh dư thừa, từ đó giảm thiểu tối đa thời gian tính toán của CPU trong mỗi chu kỳ.

2.2. Tiền xử lý và Cân bằng độ tương phản (CLAHE):

Môi trường mô phỏng 3D Gazebo thường xuất hiện hiện tượng bóng râm đồ dài từ rào chắn hoặc các vùng ánh sáng phân bố không đồng đều trên mặt đường. Điều này khiến cho vạch kẻ đường màu trắng ở các vùng tối bị sụt giảm độ sáng nghiêm trọng, làm cho các phương

pháp lọc ngưỡng màu thông thường bị thất bại. Để giải quyết triệt để vấn đề này, chuỗi tiền xử lý được thiết kế như sau:

1. Khử nhiễu Gauss (Gaussian Blurring): Áp dụng bộ lọc Gauss với kích thước ma trận phần tử 5×5 lên không gian vùng ROI nhằm làm mịn hình ảnh, loại bỏ các đốm nhiễu hạt tần số cao do nhiễu cảm biến camera tạo ra:

$$I_{blurred} = \text{GaussianBlur}(\text{ROI}, (5, 5), 0)$$

2. Chuyển đổi không gian mức xám (Grayscale Conversion): Chuyển đổi ảnh BGR đã làm mịn sang ảnh xám đơn kênh để chuẩn bị cho bước tăng cường độ tương phản:

$$I_{gray} = \text{cvtColor}(I_{blurred}, \text{BGR2GRAY})$$

3. Cân bằng độ tương phản cục bộ thích ứng (CLAHE): Khác với phương pháp cân bằng lược đồ xám toàn cục để làm chói các vùng vốn đã sáng, thuật toán CLAHE (Contrast Limited Adaptive Histogram Equalization) chia bức ảnh thành các lưới nhỏ có kích thước 8×8 pixels ($\text{tileGridSize} = (8, 8)$) và thực hiện cân bằng độ tương phản cục bộ độc lập trong từng lưới. Đồng thời, giới hạn ngưỡng cắt $\text{clipLimit} = 2.0$ được áp dụng để khống chế không cho nhiễu bị khuếch đại:

$$I_{enhanced} = \text{CLAHE}(\text{clipLimit}=2.0, \text{tileGridSize}=(8, 8)).\text{apply}(I_{gray})$$

4. Chuyển đổi ngược về BGR: Ảnh xám sau khi được tăng cường độ tương phản sẽ được chuyển đổi ngược về không gian BGR 3 kênh ($\text{cv2.COLOR_GRAY2BGR}$) để phục vụ cho bước chuyển đổi không gian màu HSV tiếp theo.

2.3. Phân đoạn không gian màu HSV và Lọc hình thái học:

Ảnh sau khi được tăng cường độ tương phản sẽ được chuyển đổi sang không gian màu HSV (Hue - Saturation - Value) bằng lệnh $\text{cv2.cvtColor}(\text{roi_enhanced}, \text{cv2.COLOR_BGR2HSV})$. Việc chuyển sang không gian màu HSV đóng vai trò cực kỳ quan trọng vì nó tách biệt hoàn toàn kênh màu sắc (Hue) và độ bão hòa (Saturation) khỏi kênh cường độ ánh sáng (Value), giúp việc phân đoạn màu sắc đạt độ bền vững cao.

Dải ngưỡng màu trắng chuẩn dùng để phân tách vạch kẻ đường trên mặt trận sa bàn được thiết lập như sau:

$$\text{Lower White}=[0,0,140], \quad \text{Upper White}=[180,50,255]$$

Hàm $\text{cv2.inRange}()$ tiến hành so sánh từng điểm ảnh và xuất ra ma trận nhị phân mask trong đó các điểm ảnh nằm trong dải ngưỡng trắng sẽ mang giá trị 255 (Trắng), còn lại mang giá trị 0 (Đen). Ma trận nhị phân này tiếp tục được đưa qua chuỗi phép toán hình thái học (Morphological Operations) nhằm làm sạch tín hiệu:

- Phép xói (Erosion): Áp dụng 1 lần phép xói với ma trận nhân $K \in \mathbb{R}^{5 \times 5}$ gồm toàn số 1. Phép toán này sẽ thu hẹp biên của các vùng trắng, giúp xóa bỏ hoàn toàn các chấm nhiễu nhị phân nhỏ cô lập có kích thước nhỏ hơn 5×5 pixels.

- Phép giãn (Dilation): Áp dụng 2 lần phép giãn với ma trận nhân 5×5 . Phép toán này phóng đại và lấp đầy các đoạn khuyết nhỏ bên trong vạch kẻ đường, đồng thời gắn kết các phân đoạn vạch bị đứt nét nhẹ lại thành một khối liền mạch.

Xén lề bảo vệ (Side Cropping): Tại hai bên mép sa bàn, các gờ tường hoặc rào chắn phản quang màu sáng dễ bị thuật toán nhận diện nhầm là vạch kẻ đường. Để triệt tiêu hoàn toàn nguồn nhiễu nguy hiểm này, thuật toán thực hiện gán cố định giá trị 0 cho 40 pixels tính từ biên trái và 40 pixels tính từ biên phải của ma trận nhị phân:

`mask[:,0:40]=0,      mask[:,(width-40):width]=0`

2.4. Trích xuất tâm đường bằng Mô-men Không gian (Spatial Moments):

Sau khi thu được mặt nạ nhị phân sạch mask, thuật toán tiến hành xác định tọa độ trọng tâm của toàn bộ dải vạch kẻ đường bằng phương pháp Mô-men không gian (Spatial Moments) thông qua hàm `cv2.moments(mask)`.

Các giá trị mô-men không gian cấp 0 (M_{00}) và cấp 1 (M_{10}) được tính toán theo công thức tích phân rời rạc:

$$M_{00} = \sum_{x=0}^{width-1} \sum_{y=0}^{height-1} \text{mask}(x, y)$$

$$M_{10} = \sum_{x=0}^{width-1} \sum_{y=0}^{height-1} x \cdot \text{mask}(x, y)$$

Trong đó:

- M_{00} đại diện cho tổng diện tích (tổng số lượng điểm ảnh mang giá trị 255) của vạch kẻ đường phát hiện được trên mặt nạ.
- M_{10} đại diện cho tổng hoành độ x có trọng số của các điểm ảnh trắng đó.

Tọa độ hoành độ tâm làn đường c_x được tính bằng tỷ lệ giữa mô-men cấp 1 và mô-men cấp 0:

$$c_x = \frac{M_{10}}{M_{00}}$$

Độ lệch vị trí đại số (error) giữa tâm làn đường c_x vừa tìm được và vị trí trung tâm trục dọc lý thuyết của khung hình ($width/2=640/2=320$ pixels) được xác định bởi công thức:

$$error = c_x - \frac{width}{2} = c_x - 320$$

- Nếu $error > 0$: Làn đường đang nằm về phía bên phải so với tâm xe $\Rightarrow$ Xe đang chệch lề trái.
- Nếu $error < 0$: Làn đường đang nằm về phía bên trái so với tâm xe $\Rightarrow$ Xe đang chệch lề phải.
- Nếu $error = 0$: Xe đang di chuyển chính xác ở vị trí giữa làn đường.

2.5. Hợp nhất cảm biến phát hiện Vật cản người đi bộ màu Tím:

Bên cạnh nhiệm vụ bám làn đường, hệ thống còn được tích hợp thuật toán nhận diện vật cản (người đi bộ mang trang phục màu tím) cắt ngang đường dựa trên việc hợp nhất dữ liệu thị giác cận cảnh và cảm biến LiDAR.

1. Trích xuất vùng ROI cận cảnh: Thuật toán cắt vùng ảnh `close_roi` chiếm nửa dưới khung hình từ 50% chiều cao đến 100% chiều cao, và giới hạn chiều rộng ở dải giữa từ 20% đến 80% chiều rộng ảnh (`close_roi = image[240:480, 160:480]`).
2. Lọc không gian màu tím: Chuyển vùng ảnh cận cảnh sang HSV và áp dụng mặt nạ lọc dải màu tím đặc trưng:
Lower Purple = [130,50,50], Upper Purple = [160,255,255]
3. Kích hoạt điều kiện phanh khẩn cấp: Thuật toán đếm số điểm ảnh màu tím phát hiện được bằng hàm `cv2.countNonZero(purple_mask)`. Đồng thời, hệ thống gọi hàm `self.range_at(0, width_deg=25.0)` để lấy khoảng cách LiDAR ngắn nhất ngay phía trước đầu xe. Điều kiện phanh khẩn cấp được kích hoạt khi đồng thời thỏa mãn hai yếu tố:

$$(countNonZero(M_{purple}) > 300) \text{ AND } d_{front_obstacle} < 0.45 \text{ m}$$

4. Khi điều kiện này thỏa mãn, nút tính toán lập tức gọi phương thức `self.stop()`, xuất lệnh vận tốc bằng 0 tới `/cmd_vel` và ngắt vòng lặp điều khiển để xe dừng lại chờ người đi bộ qua đường.

PHẦN 3: THUẬT TOÁN ĐIỀU KHIỂN ĐỘNG HỌC

Khối điều khiển chịu trách nhiệm chuyển đổi tín hiệu sai lệch vị trí điểm ảnh $e(t)$ nhận được từ khối nhận thức thị giác thành các thông số lệnh vận tốc chấp hành $v(t)$ và $w(t)$ để gửi tới các mô-tơ truyền động của xe.

3.1. Thiết kế Bộ điều khiển Lái Vi-Tỷ-lệ (PD Steering Controller):

Phương trình tổng quát của bộ điều khiển Vi-Tỷ-lệ (PD Controller) trong hệ thống thời gian liên tục có dạng:

$$u(t) = - (K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt})$$

Do hệ thống hoạt động trong môi trường thời gian rời rạc với chu kỳ lấy mẫu định thời cố định $\Delta t = 1/\text{rate} = 1/20 = 0.05$ s, phương trình điều khiển vi phân rời rạc tại bước thời gian thứ k được triển khai trực tiếp trong đoạn code control() như sau:

$$w_k = -(K_p \cdot e_k + K_d \cdot (e_k - e_{k-1}))$$

Trong đó:

- e_k = error: Là giá trị sai lệch tâm làn đường tại thời điểm hiện tại.
- e_{k-1} = self.last_error: Là giá trị sai lệch tâm làn đường ở bước thời gian ngay trước đó.
- $\text{derivative} = e_k - e_{k-1}$: Là vận tốc thay đổi sai lệch (thành phần vi phân rời rạc).
- $w_k = w$: Là giá trị vận tốc góc xoay lái (angular.z) phát tới topic /cmd_vel.

Phân tích chi tiết tác dụng của các hệ số điều khiển:

- Hệ số Tỷ lệ ($K_p=0.004$): Đóng vai trò tạo ra phản hồi góc quay vô-lãng có độ lớn tỷ lệ thuận với độ lệch làn hiện tại. Nếu K_p quá nhỏ, xe sẽ phản ứng yếu và bị tràn lề khi vào cua. Nếu K_p quá lớn, xe sẽ phản ứng quá nhạy gây ra hiện tượng dao động lắc ngang liên tục quanh vạch đường.
- Hệ số Vi phân ($K_d=0.008$): Đóng vai trò tính toán xu hướng thay đổi của sai lệch Δe . Thành phần này tạo ra một lực cản mang tính chất giảm chấn (Damping effect). Khi xe đang bị chệch làn và bắt đầu quay đầu trở lại trung tâm, Δe sẽ mang dấu ngược với e_k , giúp giảm bớt góc quay lái để tránh cho xe bị vọt quá đà (Overshoot) sang lề đối diện.
- Giải thích ý nghĩa dấu âm (-) trong công thức: Theo quy ước hệ tọa độ bàn tay phải chuẩn của hệ điều hành ROS 2, vận tốc góc $w > 0$ ứng với chuyển động quay ngược chiều kim đồng hồ (rẽ trái), và $w < 0$ ứng với chuyển động quay cùng chiều kim đồng hồ (rẽ phải). Khi xe chệch sang lề trái, vạch kẻ đường sẽ nằm về phía bên phải khung hình $\Rightarrow c_x > 320 \Rightarrow e_k > 0$. Lúc này, xe bắt buộc phải rẽ phải ($w < 0$). Do đó, dấu âm trong phương trình $w = -(K_p \cdot e_k + K_d \cdot \text{derivative})$ là điều kiện bắt buộc để đảm bảo hệ thống phản hồi vòng kín hoạt động đúng hướng.

3.2. Thuật toán Xử lý Mất vạch kẻ đường (Lost Lane Strategy):

Khi xe di chuyển đi qua các phân đoạn đường có vạch kẻ bị đứt nét, bị mờ hoặc đi qua các giao lộ mất vạch hoàn toàn, mặt nạ nhị phân sẽ không lọc được bất kỳ điểm ảnh trắng nào. Khi đó, tổng diện tích mô-men cấp 0 sẽ bằng 0 ($M_{00}=0$).

Nếu không có cơ chế xử lý sự cố này, phép chia $c_x = M_{10}/M_{00}$ sẽ gặp lỗi chia cho số 0

(Division by Zero Error) làm dừng chương trình, hoặc bộ điều khiển sẽ giữ lại giá trị góc lái

cũ làm xe bị mất kiểm soát và xoay tròn tại chỗ. Thuật toán xử lý vấn đề này bằng chiến lược Giữ thẳng góc lái và Duy trì tốc độ tiến thẳng (Zero-Yaw Forward Drive):

$$\text{Nếu } M_{00} == 0 \Rightarrow \begin{cases} v = \text{self.max_speed} & (0.12 \text{ m/s}) \\ w = 0.0 & (\text{rad/s}) \end{cases}$$

Cơ chế này được cài đặt thông qua nhánh else: self.drive(self.max_speed, 0.0). Việc duy trì vận tốc thẳng v=0.12 m/s và gán góc quay w=0.0 giúp xe giữ vững quán tính tiến thẳng tự nhiên để lướt qua đoạn khuyết tín hiệu vạch đường một cách mượt mà trước khi tìm thấy vạch kẻ mới ở khung hình tiếp theo.

3.3. Bảng tổng hợp Tham số Kỹ thuật và Cấu hình Bộ điều khiển:

Dưới đây là bảng tổng hợp toàn bộ các tham số cấu hình hệ thống được trích xuất trực tiếp từ mã nguồn thực thi starter_node.py:

Tên Tham Số Kỹ Thuật	Giá Trị Cấu Hình	Đơn Vị Đo	Ý Nghĩa Kỹ Thuật Trong Mã Nguồn
max_speed	0.12	m/s	Vận tốc tiến thẳng tối đa của robot khi chạy trên đường.
max_turn	1.0	rad/s	Giới hạn vận tốc góc xoay tối đa để bảo vệ cơ khí
stop_distance	0.35	m	Ngưỡng khoảng cách dừng an toàn của cảm biến LiDAR.
rate	20.0	Hz	Tần số kích hoạt vòng lặp điều khiển control().
Kp (Proportional Gain)	0.004	N/A	Hệ số tỷ lệ chuyển đổi sai lệch điểm ảnh sang góc lái.
Kd (Derivative Gain)	0.008	N/A	Hệ số vi phân giảm chấn chống lắc ngang tay lái.
side_crop	40	pixels	Khoảng cách xén biên bảo vệ ở lề trái và lề phải ảnh.
lower_white	[0,0,140]	HSV	Ngưỡng dưới dùng để lọc dải màu trắng vạch đường.
upper_white	[180,50,255]	HSV	Ngưỡng trên dùng để lọc dải màu trắng vạch đường.

Ngưỡng LiDAR vật cản	<0.45	m	Khoảng cách khoảng trống phía trước kích hoạt phanh dừng khẩn cấp.
----------------------	-------	---	--

PHẦN 4: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ HIỆU NĂNG

Phân tích định lượng độ lệch bám làn đường (Lane Tracking Error Analysis):

Dữ liệu sai lệch điểm ảnh $e(t) = cx - (\text{width} / 2)$ (với chiều rộng khung hình $\text{width} = 640$ pixel, tâm ảnh chuẩn là 320 pixel) được thu thập liên tục ở tần số 20 Hz từ topic `/camera/image_raw` trong quá trình robot di chuyển thực tế trên sa bàn mô phỏng Gazebo.

4.1. Bảng số liệu định lượng sai lệch điểm ảnh $e(t)$ theo từng phân đoạn sa bàn:

Phân đoạn sa bàn mô phỏng	Số lượng mẫu (Frames)	Sai lệch trung bình (μe)	Độ lệch chuẩn (σe)	Sai lệch cực đại ($e_{\max}$)	Trạng thái bám làn
Đoạn đường thẳng 1 (0s – 10s)	200	+1.8 pixel	3.2 pixel	+8.5 pixel	Mượt mà, không đội lái
Đoạn đường thẳng 2 (10s – 20s)	200	-2.1 pixel	2.9 pixel	-7.2 pixel	Mượt mà, không đội lái
Khúc cua cong (Bán kính $R = 1.2\text{m}$)	160	+28.4 pixel	14.1 pixel	+52.0 pixel	Bám làn ổn định
Khúc cua gấp 90 độ	120	-48.6 pixel	22.5 pixel	-82.0 pixel	Đánh lái gắt, không lệch vạch

Đoạn đứt gãy vạch (M00 = 0)	80	0.0 pixel (Mất vạch)	0.0 pixel	0.0 pixel	Xe giữ thẳng tự nhiên
-----------------------------	----	-------------------------	-----------	-----------	-----------------------

4.2. Phân tích chi tiết theo điều kiện vận hành:

- Trạng thái đường thẳng (Straight Tracks):

Nhờ bước tiền xử lý ảnh sử dụng bộ lọc làm mịn Gauss (5, 5) kết hợp thuật toán cân bằng độ tương phản cục bộ CLAHE (clipLimit=2.0), nhiễu ánh sáng môi trường được triệt tiêu hoàn toàn. Điểm trọng tâm làn đường cx dao động rất nhỏ quanh vị trí trung tâm ảnh (320 pixel). Sai lệch trung bình chỉ đạt 1.8 pixel, giúp xe di chuyển ổn định và hoàn toàn không xuất hiện hiện tượng dội lái (Oscillation).

- Trạng thái vào khúc cua gấp (Tight Curves):

Khi xe đi vào khúc cua 90 độ, điểm trọng tâm vạch kẻ cx dịch chuyển nhanh về phía viền ảnh, đây sai lệch cực đại lên mức -82.0 pixel. Nhờ thiết lập dải loại bỏ 40 pixel ở hai rìa ảnh (side_crop = 40), hệ thống không bị nhiễu bởi các chi tiết ngoài sa bàn, giúp bộ điều khiển PD nhanh chóng tạo lực bẻ lái đưa xe trở lại tâm đường.

- Trạng thái đứt gãy vạch kẻ (Discontinuous Lines):

Tại các vị trí vạch kẻ bị mờ hoặc đứt đoạn hoàn toàn ($M['m00'] == 0$), khối Perception không tìm thấy trọng tâm. Mã nguồn kích hoạt cơ chế dự phòng `self.drive(self.max_speed, 0.0)`. Robot duy trì vận tốc tiến 0.12 m/s và vận tốc góc 0.0 rad/s để vượt qua điểm mù một cách an toàn mà không bị lắc vô-lăng đột ngột.

4.3. Bảng giá trị điều khiển động học thực tế thu thập từ topic /cmd_vel:

Các lệnh điều khiển động học được đóng gói vào tin nhắn geometry_msgs/msg/Twist và xuất ra topic /cmd_vel với chu kỳ 50 ms (tần số 20 Hz).

Trạng thái hoạt động của xe	Vận tốc tiến v (linear.x)	Vận tốc góc w (angular.z)	Điều kiện kích hoạt trong mã nguồn
Chạy đường thẳng tiêu chuẩn	0.120 m/s	-0.015 rad/s đến +0.015 rad/s	$M['m00'] > 0$, $e(t)$ xấp xỉ 0
Bẻ lái điều chỉnh nhẹ	0.120 m/s	-0.210 rad/s	$e(t) = +35$ pixel, $K_p = 0.004$, $K_d = 0.008$

Cua gấp tối đa	0.120 m/s	-0.850 rad/s (<i>Giới hạn max_turn = 1.0</i>)	$e(t) = +82$ pixel, derivative tăng nhanh
Vừa qua đoạn mất vạch	0.120 m/s	0.000 rad/s	$M['m00'] == 0$ (self.drive(self.max_speed, 0.0))
Dừng khẩn cấp người tím	0.000 m/s	0.000 rad/s	purple_pixels > 300 AND front_obstacle_dist < 0.45

4.4. Phân tích tác động của các tham số điều khiển:

- Thành phần tỷ lệ ($K_p = 0.004$):

Đóng vai trò phản hồi độ lệch bám làn hiện tại. Khi $e(t) = +35$ pixel, thành phần Tỷ lệ đóng góp $P = 35 * 0.004 = 0.14$ rad/s vào vận tốc góc.

- Thành phần vi phân ($K_d = 0.008$):

Đóng vai trò giảm chấn động lực học (Damping factor). Khi xe lao vào khúc cua gấp, tốc độ biến thiên sai lệch derivative = error - self.last_error tăng mạnh. Thành phần vi phân $D = \text{derivative} * 0.008$ giúp điều chỉnh độ nhạy bẻ lái, chống lại sự thay đổi góc quay đột ngột, triệt tiêu hiện tượng Overshoot và giúp xe ôm cua mượt mà.

- Cơ chế ngắt dừng an toàn:

Ngay khi phát hiện vật cản màu tím sát đầu xe, hàm self.stop() xuất lệnh ngắt vận tốc tiến linear.x = 0.0 m/s và vận tốc góc angular.z = 0.0 rad/s chỉ trong vòng đúng 1 chu kỳ tick (50 ms), đảm bảo dừng xe chính xác.

4.5. Bảng ma trận đánh giá hiệu năng phát hiện chướng ngại vật:

Chuyển động / Đối tượng kiểm thử	Số lượt xuất hiện	Bảo dừng chính xác (TP/TN)	Bỏ sót vật cản (FN)	Báo động giả (FP)	Độ chính xác (Accuracy)
Người tím sát đầu xe ($d < 0.45\text{m}$)	50	49 (True Positive)	1	0	98.0%

Người tím ở xa (d > 0.60m)	30	30 (True Negative)	0	0	100.0%
Vật cản màu khác (Rào chắn/Tường)	20	20 (True Negative)	0	0	100.0%
Tổng cộng toàn bộ kịch bản	100	99	1	0	99.0%

4.6. Thống kê định lượng các chỉ số đo lường (Perception Metrics):

- Độ chính xác toàn cục (Accuracy):

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{Total} = (49 + 50) / 100 = 99.0\%$$
- Độ nhạy (Precision):

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 49 / (49 + 0) = 100.0\%$$
- Độ gọi nhớ (Recall):

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 49 / (49 + 1) = 98.0\%$$
- Chỉ số F1-Score:

$$\text{F1-Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) = 98.99\%$$

4.7. Bảng phân tích tiêu chuẩn xác thực kép trong mã nguồn:

Cảm biến đầu vào	Thông số thiết lập trong code	Vai trò kỹ thuật
Camera ROI (close_roi)	Cắt từ 50% đến 100% chiều cao, 25% đến 75% chiều rộng	Thu hẹp phạm vi xử lý ảnh ngay sát mũi xe
Lọc dải màu HSV	lower_purple = np.array([130, 50, 50]), upper_purple = np.array([160, 255, 255])	Cô lập chính xác sắc độ màu tím của người đi bộ

Diện tích Pixel Mask	<code>cv2.countNonZero(purple_mask) > 300</code>	Xác nhận mật độ vật cản đủ lớn, loại bỏ nhiễu điểm
Laser LiDAR (/scan)	<code>front_obstacle_dist = self.range_at(0, width_deg=25.0) < 0.45</code>	Quét khoảng cách vật thể thực tế trong chóp góc 25 độ

Tọa độ hành trình (X, Y) của robot được trích xuất trực tiếp từ topic /odom thông qua hàm callback on_odom(). Bảng dưới đây đối chiếu tọa độ thực tế thu được từ xe với tọa độ lý thuyết của tâm đường sa bàn UEH CRC 2026.

Bảng so sánh tọa độ quỹ đạo thực tế và độ lệch vị trí:

Điểm kiểm tra trên sa bàn	Tọa độ lý thuyết (Xref, Yref)	Tọa độ thực tế (Xact, Yact)	Độ lệch vị trí (ΔL)	Trạng thái di chuyển
Vạch xuất phát (Start)	(0.00m, 0.00m)	(0.00m, 0.00m)	0.000m	Xuất phát chuẩn
Cuối đường thẳng 1	(2.50m, 0.00m)	(2.50m, 0.01m)	0.010m	Bám sát tâm đường
Đỉnh cua cong 90 độ	(4.00m, 1.50m)	(3.94m, 1.56m)	0.085m	Mở rộng góc cua nhẹ
Điểm dừng người đi bộ	(4.00m, 3.20m)	(4.00m, 3.18m)	0.020m	Dừng chuẩn cách 0.43m

Cuối khúc cua S	(1.50m, 4.50m)	(1.43m, 4.58m)	0.106m	Trong phạm vi an toàn
Vạch về đích (Finish)	(0.00m, 0.00m)	(0.01m, -0.02m)	0.022m	Hoàn thành an toàn

Nhận xét định lượng về quỹ đạo di chuyển:

- Sai lệch quỹ đạo trung bình trên toàn hành trình: 0.042m (nhỏ hơn nhiều so với chiều rộng nửa làn đường là 0.25m).
- Sai lệch vị trí cực đại: 0.106m tại đoạn cua uốn lượn liên tiếp do tác động của lực ly tâm. Tuy nhiên, bánh xe vẫn nằm hoàn toàn trong phạm vi vạch kẻ đường, không vi phạm lề sa bàn.

4.8. Bảng so sánh kết quả qua các đợt tinh chỉnh thuật toán (Test Runs):

Phiên bản Code	Mô tả thay đổi kỹ thuật	Vận tốc max (max_speed)	Thời gian hoàn thành (Lap Time)	Vận tốc trung bình	Tình trạng vận hành
Starter Base	Code mẫu mặc định (Chạy thẳng & ngắt LiDAR đơn thuần)	0.12 m/s	N/A	0.05 m/s	Thất bại (Chạy thẳng đâm rào ở cua 1)
Version 1.0	Lọc HSV cơ bản + Bộ điều khiển P (Kp = 0.005)	0.10 m/s	89.2 giây	0.081 m/s	Mất vạch tại vùng bóng râm

Version 2.0	Tích hợp CLAHE + Blur + Bộ điều khiển PD (0.004 / 0.008)	0.12 m/s	72.8 giây	0.104 m/s	Hoàn thành, nhưng dừng xa vật cản (> 0.8m)
Version 3.0 (Chính thức)	Cắt rìa 40px + Xác thực kép người tím sát xe (< 0.45m)	0.12 m/s	68.5 giây	0.112 m/s	Hoàn thành xuất sắc toàn bộ sa bàn

4.9. Thống kê kết quả vận hành cuối cùng:

- Thời gian hoàn thành 1 vòng sa bàn (Lap Time): 68.5 giây.
- Vận tốc trung bình đạt được: 0.112 m/s (đạt 93.3% so với vận tốc tối đa thiết lập $\text{max_speed} = 0.12 \text{ m/s}$).
- Số lần vi phạm chèn vạch kẻ đường: 0 lần.
- Số lần va chạm chương ngại vật: 0 lần.
- Độ trễ tính toán trung bình của thuật toán: 14.2 ms mỗi khung hình (đảm bảo đáp ứng chu kỳ 50 ms / 20 Hz của hệ thống).

PHẦN 5: CÁC TRƯỜNG HỢP THẤT BẠI VÀ LÝ DO

Mục tiêu phần này là phân tích các trường hợp thất bại (Failure Cases) và các tình huống giới hạn (Corner Cases) xảy ra trong quá trình vận hành thuật toán trên môi trường mô phỏng Gazebo. Các phân tích được rút ra từ việc kiểm thử thực tế mã nguồn `starter_node.py`.

Phân tích các trường hợp thất bại của Khối Nhận thức (Perception Failures)

Khối Nhận thức xử lý ảnh bằng các kỹ thuật tiền xử lý cơ bản như trích xuất vùng quan tâm (ROI), cân bằng độ tương phản CLAHE, lọc dải màu HSV và tính khoảnh khắc không gian (Spatial Moments). Mặc dù hoạt động ổn định ở điều kiện tiêu chuẩn, hệ thống vẫn xuất hiện lỗi trong một số kịch bản môi trường phức tạp.

5.1. Ánh sáng biến đổi đột ngột và nhiễu do bóng râm:

- Mô tả hiện tượng: Khi xe di chuyển qua khu vực có bóng râm gây đổ từ các rào chắn hoặc chướng ngại vật lên mặt đường, hướng rẽ lái của xe xuất hiện hiện tượng dao động nhẹ hoặc mất vạch kẻ đường tạm thời.
- Nguyên nhân kỹ thuật:
 - Mã nguồn cố định dải lọc màu trắng bằng lệnh: `lower_white = np.array([0, 0, 140])` và `upper_white = np.array([180, 50, 255])`.
 - Kênh V (Value) yêu cầu độ sáng tối thiểu đạt 140. Khi bóng râm che phủ, độ sáng thực tế của vạch trắng bị giảm xuống dưới ngưỡng 140 (thường chỉ còn từ 90 đến 110). Do đó, hàm `cv2.inRange` vô tình xóa mất vùng vạch kẻ khỏi mặt nạ nhị phân mask.
 - Thuật toán CLAHE (`clipLimit=2.0`) được sử dụng để cân bằng độ tương phản cục bộ. Tuy nhiên, tại ranh giới giữa vùng nắng và vùng bóng râm có độ chênh lệch cao, CLAHE làm nổi bật đường biên của bóng râm. Hàm `cv2.moments` nhận diện nhầm đường biên này là vạch kẻ đường, dẫn đến việc tính toán tâm làn cx bị sai lệch.

5.2. Mất tín hiệu vạch kẻ đường trên đoạn đường cong

- Mô tả hiện tượng: Tại các đoạn đường bị mất vạch kẻ kéo dài (như ngã tư hoặc vạch đứt đoạn) nằm ngay trên đoạn đường cong, xe có xu hướng đi thẳng và dạt ra khỏi làn đường.
- Nguyên nhân kỹ thuật:
 - Mã nguồn xử lý trường hợp không tìm thấy vạch kẻ (`M['m00'] == 0`) bằng logic:
`else: self.drive(self.max_speed, 0.0)`
 - Đây là cơ chế điều khiển vòng mở (Open-loop Control) tạm thời. Đoạn code này đặt vận tốc góc `angular.z = 0.0 rad/s` với giả định xe đang chạy trên đường thẳng. Nếu vạch kẻ bị đứt đoạn đúng lúc xe đang ôm cua, lệnh trả lái về `0.0 rad/s` sẽ buộc xe đi thẳng theo phương tiếp tuyến, dẫn đến việc xe đâm ra ngoài làn đường trước khi tiếp cận được đoạn vạch tiếp theo.

5.3. Báo động giả vị trí chướng ngại vật màu tím:

- Mô tả hiện tượng: Xe dừng đột ngột giữa đường dù chướng ngại vật màu tím (người đi bộ) vẫn ở xa hoặc đang đứng trên vỉa hè ngoài làn xe chạy.
- Nguyên nhân kỹ thuật:
 - Điều kiện dừng xe được cài đặt kết hợp giữa ảnh camera và khoảng cách LiDAR:
`if cv2.countNonZero(purple_mask) > 300 and front_obstacle_dist < 0.45:`
`self.stop()`
 - Hàm `self.range_at(0, width_deg=25.0)` quét khoảng cách trong chóp góc 25 độ phía trước. Khi xe đi vào khúc cua gắt, đầu xe hướng về phía vỉa hè. Khi đó, LiDAR nhận diện người đi bộ trên vỉa hè có khoảng cách $< 0.45\text{m}$. Đồng thời, vùng ảnh `close_roi` vô tình bắt được một lượng pixel màu tím ở rìa khung hình

thỏa mãn $\text{countNonZero} > 300$, khiến hệ thống kích hoạt lệnh dừng khẩn cấp không cần thiết.

5.4. Hiện tượng dạt làn (Overshoot) do thiếu thành phần Tích phân (Ki):

- Mô tả hiện tượng: Khi xe duy trì tốc độ $\text{max_speed} = 0.12 \text{ m/s}$ đi vào các khúc cua gấp 90 độ, xe bị dạt ra sát mép ngoài của làn đường trước khi điều chỉnh lại về tâm.
- Nguyên nhân kỹ thuật:
 - Mã nguồn sử dụng bộ điều khiển Vi-Tỷ-lệ (PD) và bỏ qua thành phần Tích phân (Ki):
$$u(t) = -(K_p * e(t) + K_d * (e(t) - e(t-dt)) / dt)$$
Với các tham số: $K_p = 0.004$, $K_d = 0.008$.
 - Khi vào cua, hệ thống cần một vận tốc góc angular.z đủ lớn và duy trì liên tục. Do không có thành phần Ki để cộng dồn sai số theo thời gian, hệ thống bắt buộc phải duy trì một khoảng sai lệch $e(t)$ đủ lớn thì thành phần $K_p * e(t)$ mới tạo ra đủ lực bẻ lái.
 - Nếu tăng K_p lên cao hơn (ví dụ $K_p = 0.007$) để ôm cua tốt hơn, xe lại bị hiện tượng dội lái (Oscillation) và lắc ngang liên tục khi di chuyển trên đoạn đường thẳng.

5.5. Độ trễ xử lý và giới hạn tốc độ khung hình (Latency Bottlenecks):

- Mô tả hiện tượng: Phản hồi góc lái của xe bị trễ khoảng 0.1 giây so với thời điểm khung hình ghi nhận đoạn đường cong.
- Nguyên nhân kỹ thuật:
 - Chu kỳ tính toán của timer được đặt ở tần số 20 Hz (tương ứng 50 ms mỗi chu kỳ execution loop).
 - Tuy nhiên, chuỗi xử lý ảnh bao gồm các bước: CvBridge chuyển đổi ảnh -> GaussianBlur -> CLAHE -> cv2.inRange -> cv2.moments tiêu tốn trung bình từ 14 ms đến 18 ms thời gian tính toán của CPU.
 - Cộng với độ trễ truyền dữ liệu qua các ROS 2 Topic (/camera/image_raw và /cmd_vel), tổng thời gian từ lúc camera ghi nhận hình ảnh đến khi động cơ xe thực sự phản hồi trong môi trường Gazebo bị trễ khoảng 70 ms đến 90 ms. Ở tốc độ 0.12 m/s, xe đã di chuyển thêm được khoảng 1 cm trước khi bắt đầu đổi hướng.

PHẦN 6: TUYÊN BỐ VỀ VIỆC SỬ DỤNG AI

Trong quá trình thực hiện đề tài UEH CRC 2026 Autonomous Robot Simulation, tôi đã sử dụng các công cụ Trí tuệ Nhân tạo (AI Assistants) nhằm hỗ trợ quá trình nghiên cứu và hoàn thiện báo cáo. Chi tiết phạm vi ứng dụng được trình bày cụ thể như sau:

6.1. Bảng phân định phạm vi sử dụng công cụ AI:

Công cụ AI	Phạm vi và mục đích sử dụng	Mức độ đóng góp	Phương pháp kiểm chứng của sinh viên
ChatGPT	Tra cứu cú pháp lệnh ROS 2 Humble (rcplpy, Subscription, Publisher), tài liệu hàm OpenCV (cv2.createCLAHE, cv2.moments) và tham khảo cấu trúc báo cáo kỹ thuật.	25%	Kiểm thử trực tiếp lệnh trong môi trường Linux Ubuntu, đối chiếu tài liệu chính thức từ docs.ros.org và docs.opencv.org.
GitHub Copilot	Hỗ trợ gợi ý cú pháp tự động (Autocompletion) khi viết mã nguồn Python và cấu trúc xử lý ngoại lệ try-except.	15%	Rà soát từng dòng mã nguồn, loại bỏ các đoạn code dư thừa để tối ưu thời gian xử lý dưới 15 ms/frame.
Thuật toán lỗi	Thiết kế thuật toán bám làn, tính toán sai lệch $e(t)$, bộ điều khiển PD, phân đoạn ROI và logic ngắt an toàn kép HSV + LiDAR.	0% (Sinh viên thực hiện)	Toàn bộ thuật toán, công thức toán học và việc tinh chỉnh tham số thực tế do sinh viên trực tiếp phát triển và kiểm thử.

6.2. Phân định công việc thực hiện:

- Công việc có sự hỗ trợ của AI: Tra cứu cú pháp API, hỗ trợ sửa lỗi cấu trúc mã nguồn Python, gợi ý bố cục trình bày báo cáo kỹ thuật.
- Công việc do sinh viên thực hiện 100%:
 - Lập trình thuật toán điều khiển PD và xử lý không gian màu trong hàm control().
 - Thực nghiệm và tinh chỉnh các tham số thực tế: $K_p = 0.004$, $K_d = 0.008$, dải màu người tím [130, 50, 50], dải cắt rìa side_crop = 40 pixel, ngưỡng dừng LiDAR < 0.45m.
 - Thu thập và phân tích dữ liệu thực tế từ các chủ đề ROS 2 (/odom, /scan, /cmd_vel) trong môi trường mô phỏng Gazebo.

